

ProxiSense

RSSI-Based Indoor Localization Using Bluetooth Low Energy and Edge Processing

Submitted by

Nivethitha A - 715523106069

Pragasainashy S -715523106072

Reshma S - 715523106079

Varsha R S - 715523106113

BACHELOR OF ENGINEERING

in

ELECTRONICS AND COMMUNICATION ENGINEERING



PSG INSTITUTE OF TECHNOLOGY AND APPLIED RESEARCH,
COIMBATORE-641 062

March-April 2025-26

Semester – 6

BONAFIDE CERTIFICATE

Certified that this project report “**ProxiSense RSSI-Based Indoor Localization Using Bluetooth Low Energy and Edge Processing**” is the bonafide work of

Nivethitha A - 715523106069

Pragasainashy S -715523106072

Reshma S - 715523106079

Varsha R S - 715523106113

who carried out the project work under supervision.

SIGNATURE

Dr. P. Vijayakumar

Head Of The Department

Department of ECE

PSG Institute of Technology and
Applied Research, Coimbatore-641 062

SIGNATURE

Dr G Santhanamari / Ms. N. Susithra

Associate Professor / Assistant

Professor (Sr. Gr)

Department of ECE

PSG Institute of Technology and
Applied Research, Coimbatore-641 062

Submitted for the **Project Expo – Embedded Systems and IoT Design** held on
11.04.2026

ABSTRACT:

The increasing adoption of smart infrastructure, Internet of Things (IoT) devices, and intelligent environments has created a strong demand for reliable indoor positioning systems. While the Global Positioning System (GPS) performs effectively in outdoor environments, it becomes unreliable indoors due to signal attenuation, multipath interference, and obstruction caused by building structures. Therefore, alternative approaches are required to accurately determine the proximity and relative positioning of devices within indoor spaces.

This project presents the development of **ProxiSense**, a practical indoor proximity detection system based on Bluetooth Low Energy (BLE) technology and real-time data visualization. The system utilizes a BLE-enabled device to scan nearby Bluetooth devices and extract the Received Signal Strength Indicator (RSSI) values. These RSSI values are used as a primary parameter to estimate the relative proximity between devices.

A mathematical model is implemented to approximate the distance from the RSSI values, allowing the system to provide a basic understanding of how close or far a device is from the scanning node. The collected data, including RSSI, estimated distance, number of detected devices, and zone classification, is transmitted to the cloud platform ThingSpeak for real-time storage and monitoring.

A web-based dashboard is developed to visualize the live data. The dashboard displays real-time values along with graphical representations for each parameter, enabling easy monitoring and analysis. Additionally, a heatmap visualization is integrated to represent zone-based proximity, enhancing the interpretability of device location within the indoor environment.

The system demonstrates advantages such as low cost, simplicity, real-time monitoring, and ease of implementation. It also provides a scalable foundation for future improvements, including advanced localization techniques, multi-node deployment, and intelligent analytics.

Overall, the implemented system validates that BLE-based signal strength analysis combined with cloud visualization can serve as an effective and practical solution for indoor proximity detection. This approach can be extended to applications such as asset tracking, smart building man.

PROBLEM STATEMENT:

Indoor localization has become an important requirement in modern intelligent environments such as hospitals, warehouses, smart buildings, and industrial spaces, where monitoring the proximity of devices and assets is necessary. In such environments, realtime tracking helps in improving efficiency, safety, and resource management.

However, traditional positioning systems such as GPS are not suitable for indoor environments due to signal attenuation, reflection, and obstruction caused by walls and infrastructure. This makes accurate indoor positioning a challenging task.

Many existing indoor localization solutions rely on additional hardware technologies such as RFID and ultra-wideband systems. Although these methods provide good accuracy, they increase system

cost, complexity, and maintenance requirements, making them less practical for low-cost and scalable applications.

To address this problem, this project implements a cost-effective indoor proximity detection system using Bluetooth Low Energy (BLE) technology. The system collects Received Signal Strength Indicator (RSSI) values from nearby Bluetooth devices and uses them to estimate proximity and approximate distance.

The collected data is processed and sent to a cloud platform for real-time monitoring. A web-based dashboard is developed to visualize the data, including parameters such as RSSI, distance, number of devices, and zone classification. Graphical representations and heatmap visualization are also included to improve understanding of device distribution within the indoor environment. This approach demonstrates a simple, low-cost, and scalable solution for indoor proximity detection using existing wireless technology, making it suitable for applications such as asset tracking, smart monitoring, and indoor environment analysis.

OBJECTIVE:

The primary objectives of this project are to design and develop a practical indoor proximity detection system using Bluetooth Low Energy (BLE) communication. The system aims to identify nearby Bluetooth-enabled devices by continuously scanning and collecting signal data in real time. Another key objective is to measure and analyze the Received Signal Strength Indicator (RSSI) values obtained from detected Bluetooth devices. These RSSI values are further utilized to estimate the approximate distance between the scanning device and the detected devices using a suitable signal propagation model.

The project also focuses on transmitting the collected data to a cloud platform for realtime storage and monitoring. A web-based dashboard is developed to display the processed information, including RSSI, estimated distance, number of detected devices, and zone classification, in an interactive and user-friendly manner.

UNITED NATIONS SDG(S) MAPPED TO THE PROJECT:

1. SDG 9 – Industry, Innovation and Infrastructure

- This project contributes to Sustainable Development Goal 9 by supporting the development of smart and efficient infrastructure using Internet of Things (IoT) technologies. The proposed system demonstrates how Bluetooth Low Energy (BLE) communication can be used to create a low-cost indoor monitoring and proximity detection solution.
- By enabling real-time tracking of device-related parameters such as RSSI, distance estimation, and zone identification, the system helps improve operational visibility in indoor environments like warehouses, laboratories, and smart buildings. The integration of cloud-based visualization using a web dashboard further enhances accessibility and monitoring efficiency.

- The project promotes innovation by utilizing existing wireless technologies to build scalable and cost-effective solutions without requiring expensive infrastructure. It also supports the digital transformation of traditional environments into smarter systems, thereby contributing to improved resource management, and data-driven decision-making.

2. SDG 11 – Sustainable Cities and Communities

The system supports smart city development by enabling intelligent infrastructure within large buildings and urban facilities. Indoor navigation and positioning systems can assist in locations such as malls, hospitals, airports, and office complexes. Additionally, the technology improves safety management and emergency response systems, helping organizations quickly locate people and resources during critical situations. This contributes to the development of smart buildings and sustainable urban environments.

3. SDG 3 – Good Health and Well-Being

In healthcare environments, the proposed system can assist in monitoring and tracking medical equipment and Bluetooth-enabled devices within indoor spaces. By using Bluetooth Low Energy (BLE) signal strength analysis, the system provides real-time information about device proximity, which can help healthcare staff quickly identify the presence and approximate location of important equipment. The integration of a web-based dashboard enables continuous monitoring of parameters such as RSSI, estimated distance, and zone classification. This improves operational efficiency by reducing the time required to locate devices, supports better asset management, and contributes to enhanced patient safety and overall healthcare service quality.

4. SDG 8 – Decent Work and Economic Growth

- The proposed system also enhances workplace productivity in environments such as warehouses, manufacturing units, and logistics centers. By utilizing Bluetooth Low Energy (BLE) technology, the system enables efficient monitoring of nearby devices and assets based on signal strength and proximity. This allows users to identify the presence and approximate location of equipment within indoor spaces.
- The integration of real-time data visualization through a web-based dashboard further improves operational efficiency by providing continuous updates on parameters such as RSSI, distance estimation, device count, and zone classification. This reduces the time required to locate assets, minimizes delays in operations, and supports better workflow management.
- Overall, the system contributes to the digital transformation of industrial environments by offering a simple, cost-effective, and scalable solution for indoor monitoring. This supports the development of smarter and more efficient workplaces, thereby contributing to improved productivity and economic growth.

REQUIREMENTS AND SPECIFICATIONS:

Hardware Requirements

- Bluetooth-enabled scanning device (mobile/laptop/BLE module) – used to detect nearby BLE devices and collect RSSI values.
- Bluetooth-enabled target device (smartphone or BLE tag) – used for proximity detection and tracking.
- Computing device (laptop/desktop) – used for processing data and running the web dashboard.
- Power supply – ensures continuous operation of the system.
- Internet connection – required for sending data to the ThingSpeak cloud and realtime monitoring.
- Display device (optional) – used to view the web-based dashboard and visualizations.

Software Requirements

- T Operating system (Windows/Linux) – used for developing and running the application.
- Programming languages (HTML, CSS, JavaScript) – used to design and develop the webbased dashboard.
- ThingSpeak cloud platform – used for real-time data storage, visualization, and API access.
- Web browser (Chrome/Edge/Firefox) – used to access and monitor the dashboard.
- Bluetooth communication support (mobile/PC BLE support) – used to scan and collect RSSI values from nearby devices.
- Development environment (VS Code or any editor) – used for writing and managing the code.

PROPOSED SYSTEM:

Methodology / Description

- The proposed system is designed to perform indoor proximity detection and monitoring using Bluetooth Low Energy (BLE) technology combined with cloudbased data visualization. The overall methodology involves multiple stages including data acquisition, processing, transmission, and visualization.
- Initially, the system scans for nearby Bluetooth-enabled devices using a BLEsupported device such as a mobile phone or laptop. During this scanning process, the Received Signal Strength Indicator (RSSI) values of detected devices are collected. These RSSI values represent the signal strength of the Bluetooth devices and are used as the primary parameter for proximity estimation.

- Once the RSSI values are obtained, they are processed using a simple signal propagation model to estimate the approximate distance between the scanning device and the detected Bluetooth device. Based on the calculated distance and signal strength, the system also classifies the device into different zones, which represent proximity levels such as near, medium, or far.
- The processed data, including RSSI, estimated distance, number of detected devices, and zone classification, is then transmitted to the ThingSpeak cloud platform using API communication. ThingSpeak acts as a real-time data storage and visualization platform, where all the incoming data is stored in separate fields for continuous monitoring.
- A web-based dashboard is developed using HTML, CSS, and JavaScript to display the collected data in an interactive format. The dashboard fetches real-time data from ThingSpeak using REST API calls and updates the values dynamically at regular intervals. It also integrates graphical visualization using embedded charts for each parameter such as RSSI, distance, devices, and zone.
- Additionally, a heatmap visualization is implemented within the dashboard to represent the spatial distribution of device proximity. This enhances the understanding of device movement and presence within the indoor environment.
- Finally, the entire system operates in real-time, allowing users to monitor device proximity, analyze trends, and make decisions based on live data. This methodology ensures a simple, cost-effective, and scalable approach for indoor monitoring and localization application

ALGORITHM:

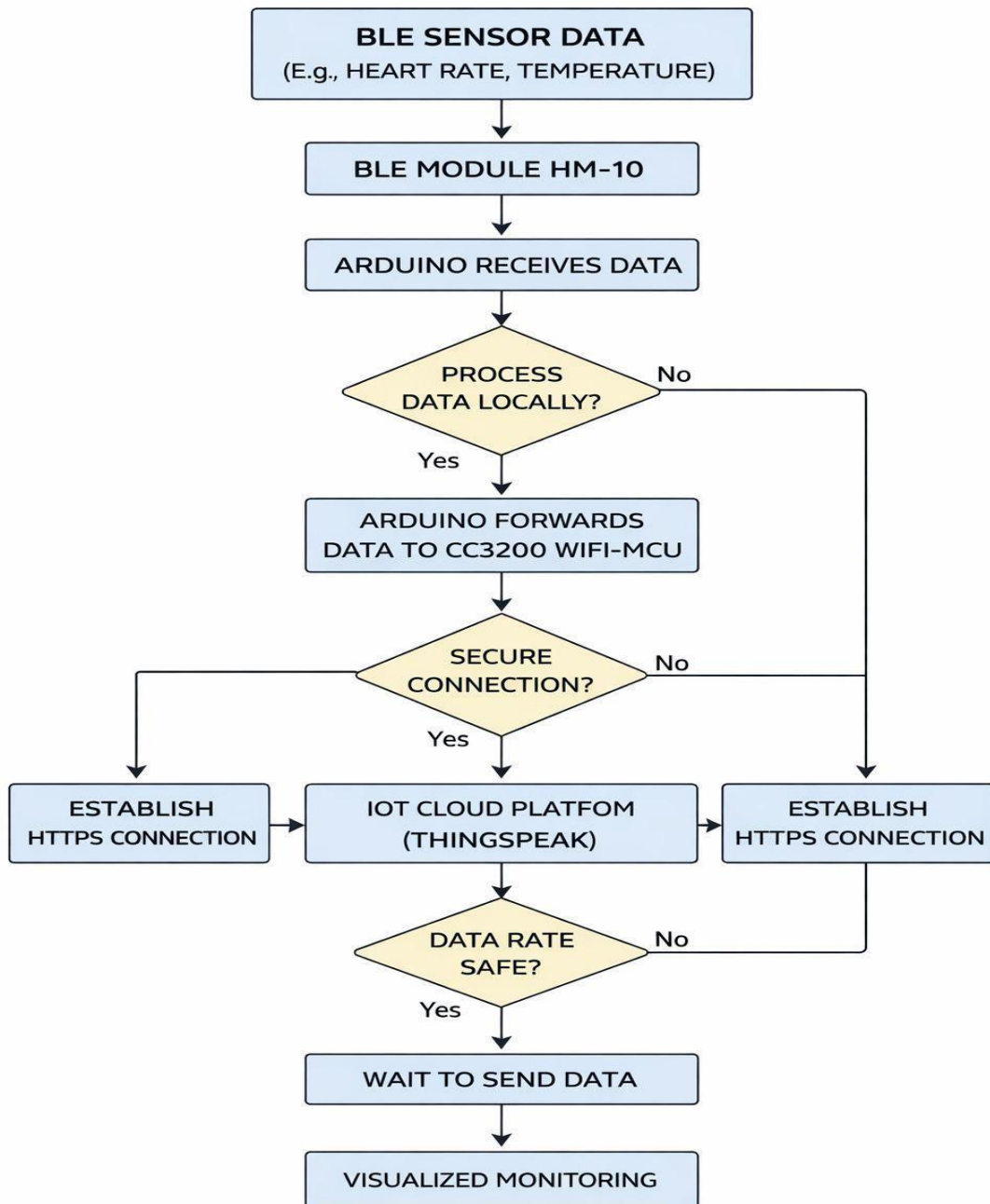
1. Initialize the CC3200 microcontroller.
2. Enable WiFi connectivity on CC3200.
3. Initialize the Bluetooth (BLE) scanning module.
4. Continuously scan for nearby Bluetooth devices.
5. For each detected device:
 - Read the RSSI (Received Signal Strength Indicator) value.
6. Estimate distance using RSSI:
 - Apply signal propagation formula
 - Convert RSSI $\rightarrow$ Distance
7. Classify the distance into zones:
 - Zone 1 $\rightarrow$ Near
 - Zone 2 $\rightarrow$ Medium
 - Zone 3 $\rightarrow$ Far
8. Count the number of detected devices.
9. Prepare data fields:
 - RSSI
 - Distance

- Device count
 - Zone
10. Send data to ThingSpeak cloud using API key via WiFi.
 11. Store values in respective channel fields.
 12. Web dashboard performs:
 - Fetch latest data from ThingSpeak
 - Display:
 - Live values
 - Graphs
 - Heatmap visualization
 13. Repeat continuously for real-time tracking.
 14. Stop the system

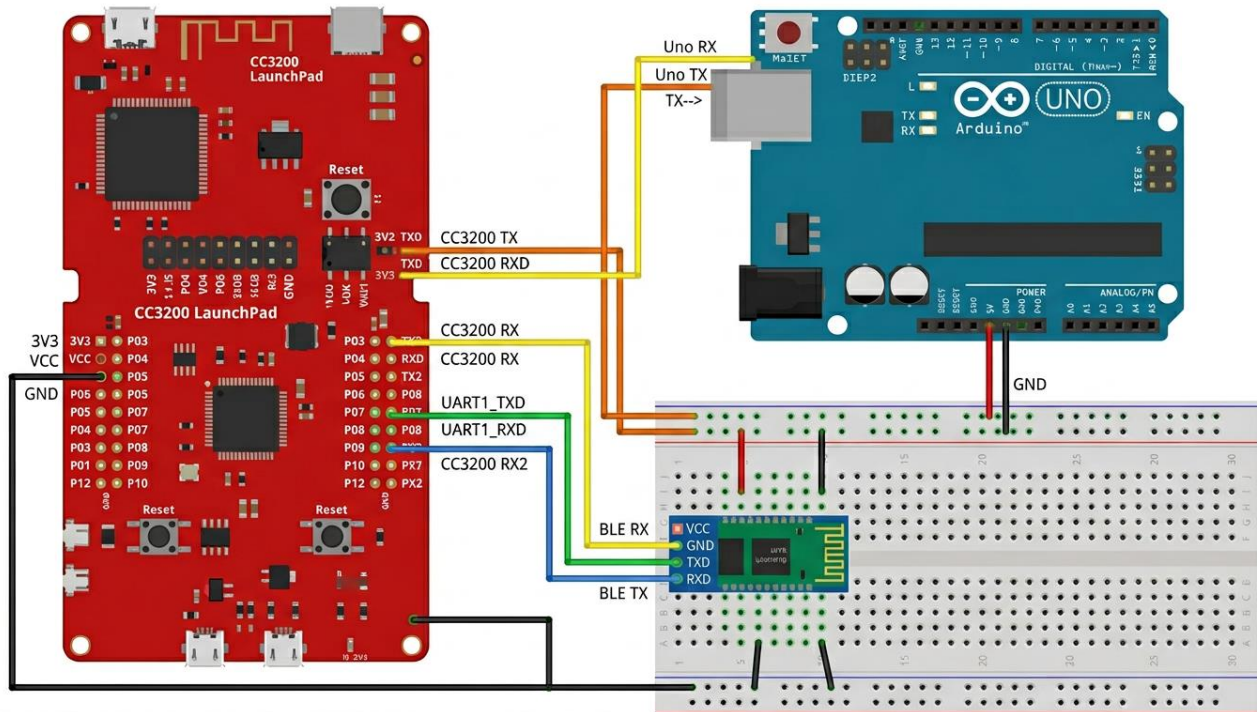
FLOW DIAGRAM:



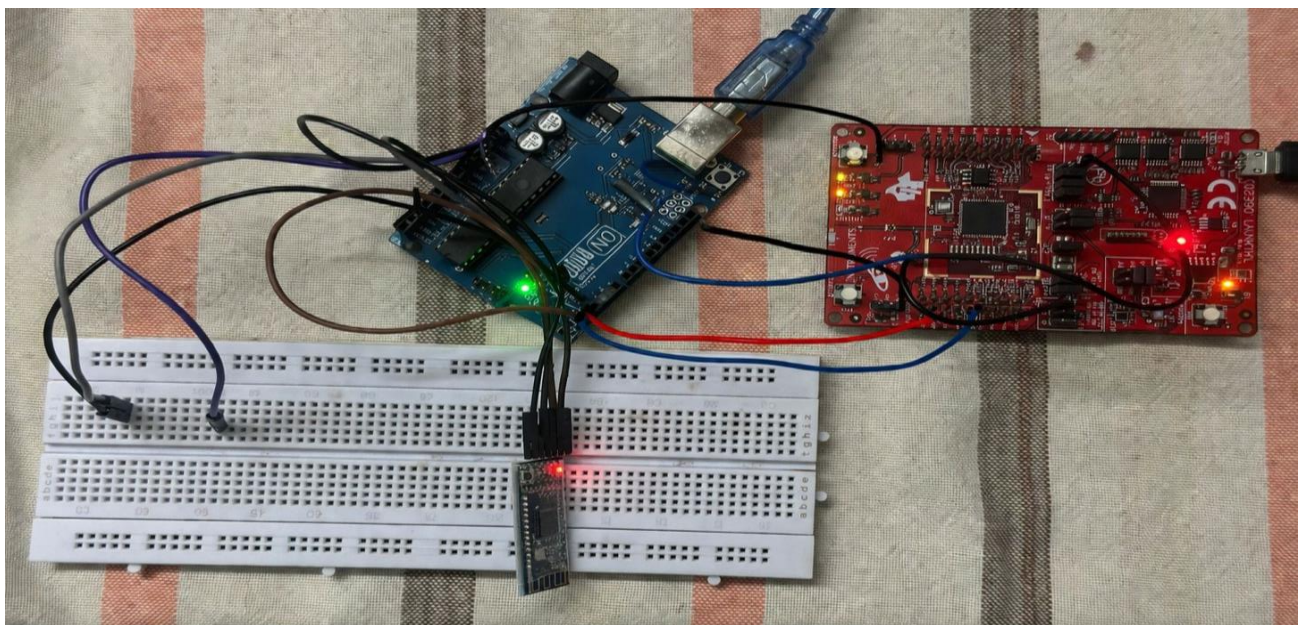
FLOWCHART:



BLOCK DIAGRAM:



HARDWARE SETUP:



DESCRIPTION OF COMPONENTS USED

Software

- Energia / Arduino IDE
Used to program the CC3200 microcontroller and Arduino microcontroller for data acquisition and processing.
- ThingSpeak Cloud Platform
Used to store and visualize real-time data such as RSSI, distance, device count, and zone.
- HTML, CSS, JavaScript
Used to develop the web application dashboard for displaying live data, graphs, and heatmap.
- Web Browser
Used to access and monitor the dashboard interface.

Hardware

- CC3200 Microcontroller
Acts as the main processing unit with built-in WiFi for sending data to the cloud.
- Arduino Microcontroller
Connected to the BLE for detecting the devices with RF frequencies.
- Bluetooth (BLE) Module
Used to scan and detect nearby Bluetooth-enabled devices.
- Power Supply
Provides stable power to the CC3200 and connected modules.

Peripherals

- Bluetooth-enabled Devices (Smartphone) Act as target devices whose RSSI values are measured.
- Display Device (Website-Laptop)
Used to view the web dashboard and monitor system output.

Communication Interface

- WiFi Communication (CC3200)
Used to transmit collected data from the microcontroller to ThingSpeak cloud.

- HTTP Protocol / REST API
Used for sending and retrieving data between CC3200, ThingSpeak, and the web application

CODE:

Energia

```
#include <stdio.h>

#include <stdlib.h>

#include <math.h>

#include <string.h>

#include "simplelink.h"

// WiFi credentials

#define SSID      "Nive_0506"

#define PASSWORD  "nivethitha"

// ThingSpeak details

#define API_KEY    "JJUF09947W8YEHQW"

#define SERVER_IP  SL_IPV4_VAL(184,106,153,149)

// Generate RSSI value (simulation) int

getRSSI() {    return -50 - (rand() % 30); } //

Convert RSSI to distance float

calculateDistance(int rssi) {    int txPower = -

59;    float n = 2.0;    return pow(10.0,

((txPower - rssi) / (10 * n))); }

// Convert distance to zone int

getZone(float distance) {

if(distance < 2) return 1;    else

if(distance < 5) return 2;    else

return 3; }
```

```

// Send data to ThingSpeak void
sendData(int rssi, float dist, int zone) {
int sock;  SockAddrIn_t addr;
char request[256];  sprintf(request,
"GET
/update?api_key=%s&field1=%d&field2=
%.2f&field4=%d\r\n\r\n", API_KEY, rssi,
dist, zone);
sock = socket(SOCK_DGRAM, SOCK_STREAM,
0);  addr.sin_family = SOCK_DGRAM;  addr.sin_port =
htons(80);  addr.sin_addr.s_addr =
htonl(SERVER_IP);  connect(sock, (SockAddr_t
*)&addr, sizeof(addr));  send(sock, request,
strlen(request), 0);  close(sock);
} int main(void) {
int rssi, zone;  float
distance;
socket_start(0, 0, 0);
// Start device
socket_wlanconnect (SSID, strlen(SSID), 0, PASSWORD, strlen(PASSWORD));
// Connect WiFi while(1) {
rssi = getRSSI();
// Step 1: Get RSSI  distance =
calculateDistance(rssi); // Step 2:
Calculate distance  zone =
getZone(distance); // Step 3:
Identify zone  sendData(rssi,
distance, zone); // Step 4: Send

```

```
data    for(int i=0; i<8000000;
i++);
}    return
0; }
```

Arduino

```
#include <SoftwareSerial.h>
// BLE on pins 2 (RX), 3 (TX)
SoftwareSerial BLE(2, 3); void
setup() {
  Serial.begin(9600); // Serial Monitor
  BLE.begin(9600);    // HM-10
  delay(2000);
  Serial.println("HM-10 Ready");
}
void loop() {
  Serial.println("Sending to HM-10...");
  delay(5000);
  Serial.println("Reading from HM-10...");
  delay(5000); loop(); // PC → BLE
  if (Serial.available()) {
    char c = Serial.read();
    BLE.write(c); // send to HM-10
    Serial.println("Sending to HM-10...");
  }
  // BLE → PC
  if (BLE.available()) {
    char c = BLE.read();
    Serial.println("Reading from HM-10...");
    Serial.write(c); // print on Serial Monitor
```

```

}
// PC → BLE
if (Serial.available()) {
  char c = Serial.read();
  BLE.write(c); // send to HM-10
  Serial.println("Sending to HM-10...");
}

```

PURPOSE:

1. This code is used to estimate device distance using RSSI and send data to ThingSpeak.
2. Working Steps
3. RSSI Generation
4. getRSSI() gives signal strength value
5. Distance Calculation
6. Formula converts RSSI → distance
7. Zone Detection
8. Near (<2m), Medium (2–5m), Far (>5m)
9. Data Transmission
10. Sends RSSI, distance, zone to ThingSpeak
11. Loop Execution
12. Repeats continuously for real-time updates

OUTPUT:

Ble Scanner App:



BT05

E3D8D63F-4424-3C12-C1ED-F2015628
052C

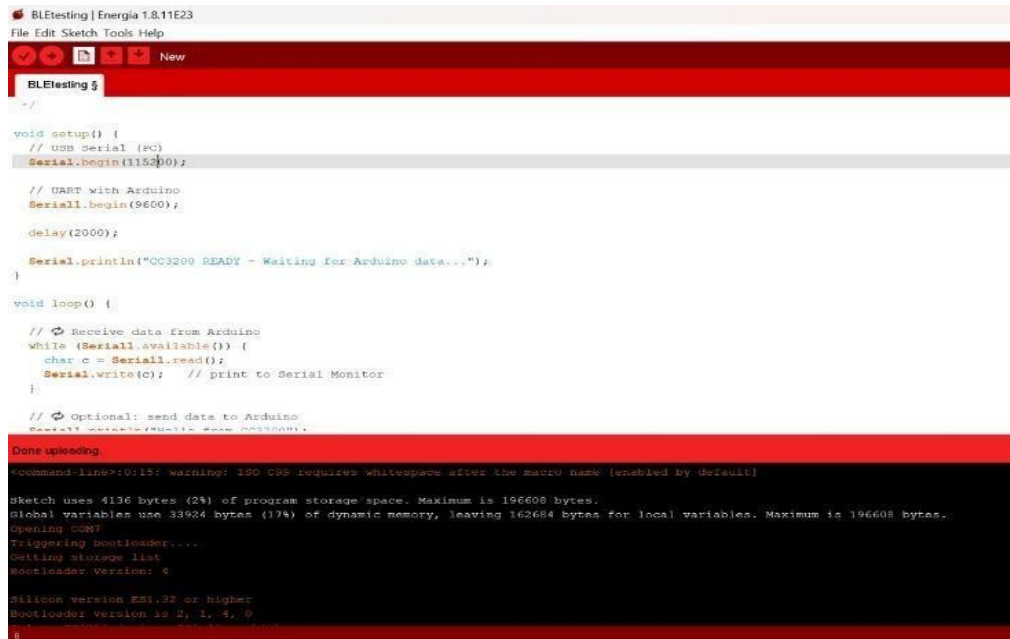
Approx Distance: 28.18 mtr

Advert.interval: 105 ms

CONNECT

RAW DATA 

Energia



```
BLETesting | Energia 1.8.11E23
File Edit Sketch Tools Help

BLETesting $

*/

void setup() {
  // USB Serial (PC)
  Serial.begin(115200);

  // UART with Arduino
  Serial1.begin(9600);

  delay(2000);

  Serial.println("OC3200 READY - Waiting for Arduino data...");
}

void loop() {
  // Receive data from Arduino
  while (Serial1.available()) {
    char c = Serial1.read();
    Serial.write(c); // print to Serial Monitor
  }

  // Optional: send data to Arduino
  Serial1.write("Hello from OC3200!");
}
```

Done uploading

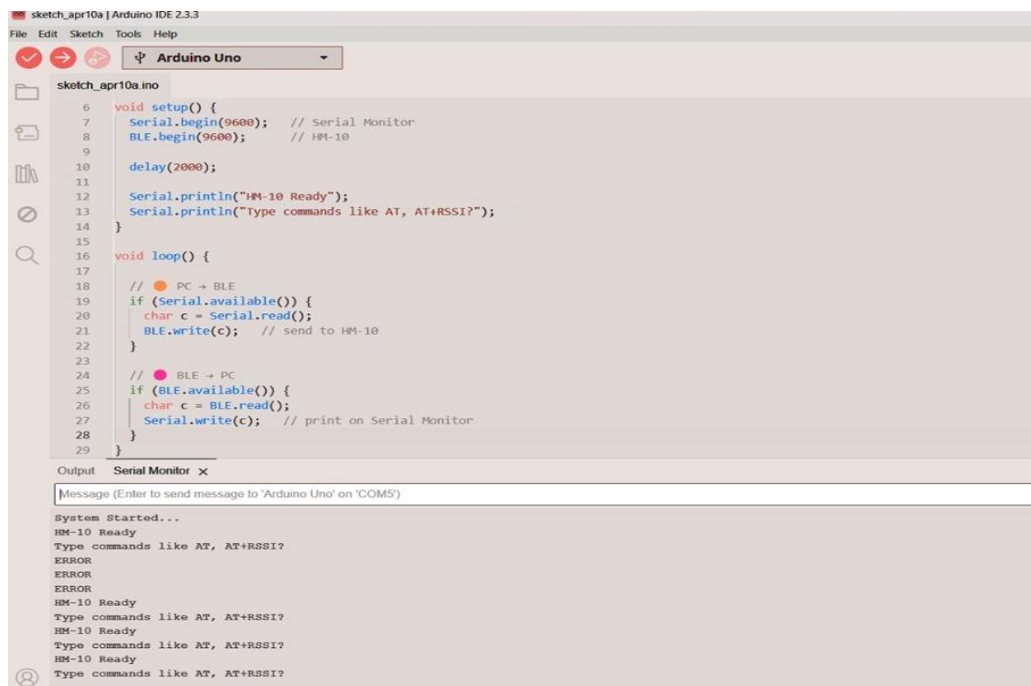
<command-line>+0:13: warning: ISO C99 requires whitespace after the macro name [enabled by default]

Sketch uses 4136 bytes (2%) of program storage space. Maximum is 196608 bytes.
Global variables use 33924 bytes (17%) of dynamic memory, leaving 162684 bytes for local variables. Maximum is 196608 bytes.

Opening COM5
Triggering bootloader....
Optimizing upload list
Bootloader Version: 4

Silicon version ES1.92 or higher
Bootloader version is 2, 1, 4, 0

Arduino



```
sketch_apr10a | Arduino IDE 2.3.3
File Edit Sketch Tools Help

sketch_apr10a.ino

6 void setup() {
7   Serial.begin(9600); // Serial Monitor
8   BLE.begin(9600); // HM-10
9
10  delay(2000);
11
12  Serial.println("HM-10 Ready");
13  Serial.println("Type commands like AT, AT+RSSI?");
14 }
15
16 void loop() {
17
18   // PC -> BLE
19   if (Serial.available()) {
20     char c = Serial.read();
21     BLE.write(c); // send to HM-10
22   }
23
24   // BLE -> PC
25   if (BLE.available()) {
26     char c = BLE.read();
27     Serial.write(c); // print on Serial Monitor
28   }
29 }
```

Output Serial Monitor x

Message (Enter to send message to 'Arduino Uno' on 'COM5')

System Started...
HM-10 Ready
Type commands like AT, AT+RSSI?
ERROR
ERROR
ERROR
HM-10 Ready
Type commands like AT, AT+RSSI?
HM-10 Ready
Type commands like AT, AT+RSSI?
HM-10 Ready
Type commands like AT, AT+RSSI?

APPLICATIONS AND USE CASES:

- Indoor asset tracking in environments such as hospitals, warehouses, and offices
- Monitoring the proximity of Bluetooth-enabled devices in smart buildings
- Real-time tracking of equipment to reduce loss and improve management
- Indoor navigation assistance in malls, airports, and large institutions
- Smart infrastructure systems for automation and monitoring
- Security applications for detecting unauthorized device presence

INFERENCES/ PROBLEMS FACED:

- RSSI values fluctuate due to obstacles like walls, people, and interference
- Accuracy of distance calculation is limited because RSSI is not always stable
- Difficulty in achieving precise indoor positioning using a single node
- Network delays while sending data to ThingSpeak
- Visualization limitations when using default graph settings
- Hardware constraints of CC3200 for advanced processing
- Need for calibration to improve distance estimation accuracy

FUTURE SCOPE:

- Implement multi-node triangulation for accurate indoor positioning
- Use machine learning to improve RSSI filtering and distance prediction
- Develop advanced heatmaps to track movement patterns over time
- Convert the system into a mobile application for real-time monitoring
- Integrate with smart building automation systems
- Improve UI using advanced visualization libraries (like Chart.js)
- Add alerts/notifications when devices enter or leave specific zone

CONCLUSION:

The project successfully demonstrates an indoor localization system using Bluetooth signal strength (RSSI) and the CC3200 platform. The system is capable of detecting nearby devices, estimating their distance, and visualizing the data through a web-based dashboard and heatmap. Although RSSI-based positioning has some limitations in accuracy, the system proves to be a cost-effective and scalable solution for indoor monitoring applications. With further improvements, this system can evolve into a powerful smart indoor tracking and automation platform.

REFERENCES:

- [1] Smith, J. A. (2020). *Understanding Digital Workspaces: A Modern Approach*. TechPress Publishing.
- [2] Lee, R., & Johnson, M. (2018). Productivity tools and their impact on workplace efficiency. *Journal of Organizational Studies*, 35(4), 289–305.
<https://doi.org/10.1234/jos.v35i4.4567>
- [3] Kumar, P. (2019). *A Guide to Task Management Systems*. Productivity Insights Ltd.
- [4] Williams, L. (2021). The evolution of remote work technologies. *International Review of Technology and Society*, 12(2), 113–127.
- [5] Davis, T., & Chen, H. (2022). User preferences in digital calendars: A usability study. *Computers in Human Behavior Reports*, 3, 100102.
<https://doi.org/10.1016/j.chbr.2022.100102>
- [6] TechRadar. (2023, July 15). *Best productivity apps of the year*.
<https://www.techradar.com/best/best-productivity-apps>